

Unit - 3

SQL: QUERIES, CONSTRAINTS, TRIGGERS: form of basic SQL query, UNION, INTERSECT, and EXCEPT, Nested Queries, aggregation operators, NULL values, complex integrity constraints in SQL, triggers and active databases.

Form of basic SQL query

The basic form of an SQL query, specifically when retrieving data, is composed of a combination of clauses. The most elementary form of an SQL query for data retrieval can be represented as

Syntax

SELECT [DISTINCT] column1, column2, ...

FROM tablename

WHERE condition;

Let's break it down:

1. **SELECT Clause:** This is where you specify the columns you want to retrieve. Use an asterisk (*) to retrieve all columns.
2. **FROM Clause:** This specifies from which table or tables you want to retrieve the data.
3. **WHERE Clause (optional):** This allows you to filter the results based on a condition.
4. **DISTINCT Clause (optional):** is an optional keyword indicating that the answer should not contain duplicates. Normally if we write the SQL without DISTINCT operator then it does not eliminate the duplicates.

Here are the primary components of SQL queries:

- **SELECT:** Retrieves data from one or more tables.

- **FROM:** Specifies the table from which you're retrieving the data.
- **WHERE:** Filters the results based on a condition.
- **GROUP BY:** Groups rows that have the same values in specified columns.
- **HAVING:** Filters the result of a GROUP BY.
- **ORDER BY:** Sorts the results in ascending or descending order.
- **JOIN:** Combines rows from two or more tables based on related columns.

To provide a more holistic view, here are a few more SQL examples, keeping them as basic as possible:

1. Retrieve all columns from a table:

Syntax

```
SELECT * FROM tablename;
```

2. Retrieve specific columns from a table:

Syntax

```
SELECT column1, column2 FROM tablename;
```

3. Retrieve data with a condition:

Syntax

```
SELECT column1, column2 FROM tablename WHERE column1 = 'value';
```

4. Sort retrieved data:

Syntax

```
SELECT column1, column2 FROM tablename ORDER BY column1 ASC;
```

Regular expressions in the SELECT Command

SQL provides support for pattern matching through the LIKE operator, along with the use of the wild-card symbols.

Regular expressions: is a sequence of characters that define a search pattern, mainly for use in pattern matching with strings, or string matching.

Examples:

Finds Names that start or ends with "a"

Finds names that start with "a" and are at least 3 characters in length.

LIKE: The LIKE operator is used in a 'WHERE' clause to search for a specified pattern in a column

wild-card: There are two primary wildcards used in conjunction with the 'LIKE' operator **percent sign (%)** Represents zero, one, or multiple characters **underscore sign(_)** Represents a single character

Here's a breakdown of how you can use these wildcards with the 'LIKE' operator:

Using `%` Wildcard

1. Find values that start with a specific pattern:

Syntax

```
SELECT column_name  
FROM table_name  
WHERE column_name LIKE 'pattern%';
```

For example, to find all customers whose names start with "Ma":

Example

```
SELECT FirstName  
FROM Customers  
WHERE FirstName LIKE 'Ma%';
```

2. Find values that end with a specific pattern:

Syntax

```
SELECT column_name  
  FROM table_name  
 WHERE column_name LIKE '%pattern';
```

For instance, to find all products that end with "ing":

Example

```
SELECT ProductName  
  FROM Products  
 WHERE ProductName LIKE '%ing';
```

3. Find values that have a specific pattern anywhere:**Syntax**

```
SELECT column_name  
  FROM table_name  
 WHERE column_name LIKE '%pattern%';
```

Example, to find all books that have the word "life" anywhere in the title:

Example

```
SELECT BookTitle  
  FROM Books  
 WHERE BookTitle LIKE '%life%';
```

Using `\_` Wildcard**1. Find values of a specific length where you only know some characters:****Syntax**

```
SELECT column_name  
  FROM table_name  
 WHERE column_name LIKE 'p_ttern';
```

For instance, if we're looking for a five-letter word where you know the first letter is "h" and the third letter is "l", you could use:

Example

```
SELECT Word
```

FROM Words

WHERE Word LIKE 'h_l_';

Combining '%' and '_'

We can use both wildcards in the same pattern. For example, to find any value that starts with "A", followed by two characters, and then "o":

Example

SELECT column_name

FROM table_name

WHERE column_name LIKE 'A__o%';

Keep in mind that the actual symbols used for wildcards might vary depending on the database system. For example, in SQL Server, the wildcard for a single character is '['?']' instead of '_'.

Always refer to the specific documentation of the database you're working with.

SQL Set Operation

The SQL Set operation is used to combine the two or more SQL SELECT statements.

Types of Set Operation

1. Union
2. Union All
3. Intersect
4. Minus/Except

1. Union

- The SQL Union operation is used to combine the result of two or more SQL SELECT queries.
- In the union operation, all the number of data type and columns must be the same in both the tables on which the UNION operation is being applied.
- The union operation eliminates the duplicate rows from its result set.

Syntax

```
SELECT column_name FROM table1
```

UNION

```
SELECT column_name FROM table2;
```

Example:

The First table

ID	NAME
1	Jack
2	Harry
3	Jackson

The Second table

ID	NAME
3	Jackson
4	Stephan
5	David

Union SQL query will be:

```
SELECT * FROM First UNION SELECT * FROM Second;
```

The result set table will look like:

ID	NAME
1	Jack

2	Harry
3	Jackson
4	Stephan
5	David

2. Union All

Union All operation is equal to the Union operation. It returns the set without removing duplication and sorting the data.

Syntax:

```
SELECT column_name FROM table1
```

```
UNION ALL
```

```
SELECT column_name FROM table2;
```

Example: Using the above First and Second table.

Union All query will be like:

```
SELECT * FROM First
```

```
UNION ALL
```

```
SELECT * FROM Second;
```

The result set table will look like:

ID	NAME
1	Jack
2	Harry
3	Jackson

3	Jackson
4	Stephan
5	David

3. Intersect

- It is used to combine two SELECT statements. The Intersect operation returns the common rows from both the SELECT statements.
- In the Intersect operation, the number of data types and columns must be the same.
- It has no duplicates and it arranges the data in ascending order by default.

Syntax

SELECT column_name FROM table1

INTERSECT

SELECT column_name FROM table2;

Example:

Using the above First and Second table.

Intersect query will be:

SELECT * FROM First

INTERSECT

SELECT * FROM Second;

The resultset table will look like:

ID	NAME
3	Jackson

4. Minus/Except

- It combines the result of two SELECT statements. Except operator is used to display the rows which are present in the first query but absent in the second query.
- It has no duplicates and data arranged in ascending order by default.

Syntax:

SELECT column_name FROM table1

Except

SELECT column_name FROM table2;

Example

Using the above First and Second table.

Except query will be:

SELECT * FROM First

Except

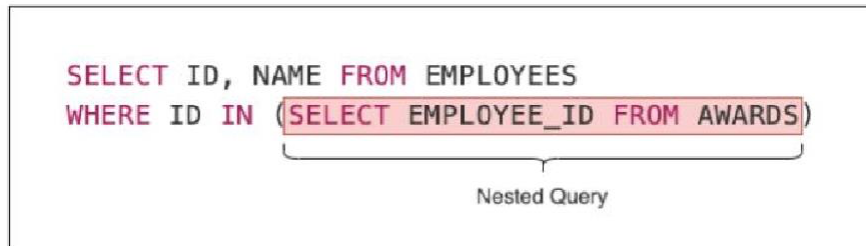
SELECT * FROM Second;

The result set table will look like:

ID	NAME
1	Jack
2	Harry

Nested Query

A nested query in SQL contains a query inside another query. The outer query will use the result of the inner query. For instance, a nested query can have two SELECT statements, one on the inner query and the other on the outer query.



Types of Nested Queries in SQL

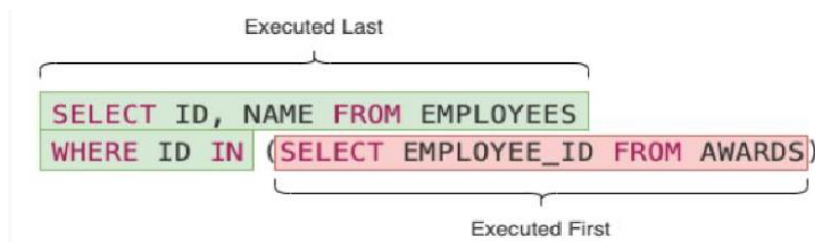
Nested queries in SQL can be classified into two different types:

- Independent Nested Queries
- Co-related Nested Queries

Independent Nested Queries

In independent nested queries, the execution order is from the innermost query to the outer query. An outer query won't be executed until its inner query completes its execution. The outer query uses the result of the inner query. Operators such as IN, NOT IN, ALL, and ANY are used to write independent nested queries.

- The IN operator checks if a column value in the outer query's result is present in the inner query's result. The final result will have rows that satisfy the IN condition.
- The NOT IN operator checks if a column value in the outer query's result is not present in the inner query's result. The final result will have rows that satisfy the NOT IN condition.
- The ALL operator compares a value of the outer query's result with all the values of the inner query's result and returns the row if it matches all the values.
- The ANY operator compares a value of the outer query's result with all the inner query's result values and returns the row if there is a match with any value.



Co-related Nested Queries

In co-related nested queries, the inner query uses the values from the outer query to execute the inner query for every row processed by the outer query. The co-related nested queries run slowly because the inner query is executed for every row of the outer query's result.

How to Write a Nested Query in SQL?

We can write a nested query in SQL by nesting a SELECT statement within another SELECT statement. The outer SELECT statement uses the result of the inner SELECT statement for processing.

The general syntax of nested queries will be:

SELECT column_name [, column_name]

FROM table1 [, table2]

WHERE column_name OPERATOR

(SELECT column_name [, column_name] FROM table1 [, table2] [WHERE]);

The SELECT query inside the brackets () is the inner query, and the SELECT query outside the brackets is the outer query. The outer query uses the result of the inner query.

Examples of Nested Query in SQL

We will use the Employees and Awards table below to understand independent and co-related nested queries. We will be using Oracle SQL syntax in our queries.

Let's create the Employees and Awards tables:

```
CREATE TABLE employee (id NUMBER PRIMARY KEY, name VARCHAR2(100) NOT NULL, salary NUMBER NOT NULL, role VARCHAR2(100) NOT NULL );
```

```
CREATE TABLE awards (id NUMBER PRIMARY KEY, employee_id NUMBER NOT NULL, award_date DATE NOT NULL);
```

Let's add data to the tables created above:

```
INSERT INTO employees VALUES (1, 'Augustine Hammond', 10000, 'Developer');
```

```
INSERT INTO employees VALUES (2, 'Perice Mundford', 10000, 'Manager');
```

```
INSERT INTO employees VALUES (3, 'Cassy Delafoy', 30000, 'Developer');
```

```
INSERT INTO employees VALUES (4, 'Garwood Saffen', 40000, 'Manager');
```

```
INSERT INTO employees VALUES (5, 'Faydra Beaves', 50000, 'Developer');
```

```
INSERT INTO awards VALUES(1, 1, TO_DATE('2022-04-01', 'YYYY-MM-DD'));
```

```
INSERT INTO awards VALUES(2, 3, TO_DATE('2022-05-01', 'YYYY-MM-DD'));
```

Employees

id	name	salary	Role
1	Augustine Hammond	10000	Developer
2	Perice Mundford	10000	Manager
3	Cassy Delafoy	30000	Developer

4	Garwood Saffen	40000	Manager
5	Faydra Beaves	50000	Developer

Awards

id	employee_id	award_date
1	1	2022-04-01
2	3	2022-05-01

Independent Nested Queries

Example 1: IN

Select all employees who won an award.

```
SELECT id, name FROM employees
```

```
WHERE id IN (SELECT employee_id FROM awards);
```

Output

id	name
2	Perice Mundford
4	Garwood Saffen
5	Faydra Beaves

Example 2: NOT IN

Select all employees who never won an award.

```
SELECT id, name FROM employees
```

```
WHERE id NOT IN (SELECT employee_id FROM awards);
```

Output

id	name
1	Augustine Hammond
3	Cassy Delafoy

Example 3: ALL

Select all Developers who earn more than all the Managers

```
SELECT * FROM employees
```

```
WHERE role = 'Developer'
```

```
AND salary > ALL (SELECT salary FROM employees WHERE role = 'Manager');
```

Output

id	name	salary	role
5	Faydra Beaves	50000	Developer

Explanation

The developer with id 5 earns (50000) more than all the managers: 2 (10000) and 4 (40000)

Example 4: ANY

Select all Developers who earn more than any Manager

```
SELECT * FROM employees
```

```
WHERE role = 'Developer'
```

```
AND salary > ANY (SELECT salary FROM employees WHERE role = 'Manager');
```

Output

id	Name	salary	role
5	Faydra Beaves	50000	Developer
3	Cassy Delafoy	30000	Developer

Explanation

The developers with id 3 and 5 earn more than any manager:

- The developer with id 3 earns (30000) more than the manager with id 2 (10000)
- The developer with id 5 earns (50000) more than the managers with id 2 (10000) and 4 (40000)

Co-related Nested Queries

Select all employees whose salary is above the average salary of employees in their role.

```
SELECT * FROM employees emp1
```

```
WHERE salary > (SELECT AVG(salary) FROM employees emp2 WHERE emp1.role = emp2.role);
```

Output

id	Name	salary	role
4	Garwood Saffen	40000	Manager
5	Faydra Beaves	50000	Developer

Explanation

The manager with id 4 earns more than the average salary of all managers (25000), and the developer with id 5 earns more than the average salary of all developers (30000). The inner query

is executed for all rows fetched by the outer query. The inner query uses the role value (emp1.role) of every outer query's row (emp1.role = emp2.role).

We can find the average salary of managers and developers using the below query:

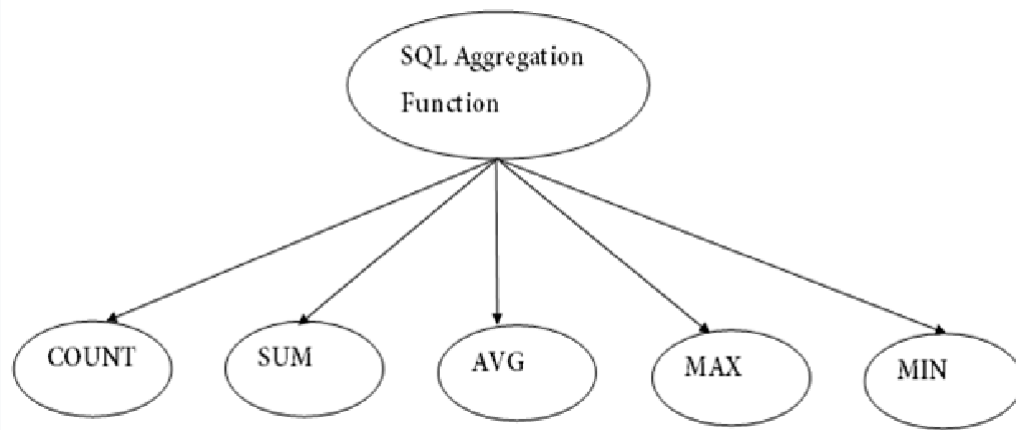
```
SELECT role, AVG(salary)
FROM employees GROUP BY role;
```

role	avg(salary)
Developer	30000
Manager	25000

SQL Aggregate Functions

- SQL aggregation function is used to perform the calculations on multiple rows of a single column of a table. It returns a single value.
- It is also used to summarize the data.

Types of SQL Aggregation Function



1. COUNT FUNCTION

- The COUNT function is used to Count the number of rows in a database table. It can work on both numeric and non-numeric data types.
- COUNT function uses the COUNT(*) that returns the count of all the rows in a specified table. COUNT(*) considers duplicate and Null.

Syntax

1. COUNT(*)
2. or
3. COUNT([ALL|DISTINCT] expression)

Sample table:

PRODUCT_MAST

PRODUCT	COMPANY	QTY	RATE	COST
Item1	Com1	2	10	20
Item2	Com2	3	25	75
Item3	Com1	2	30	60
Item4	Com3	5	10	50

Item5	Com2	2	20	40
Item6	Cpm1	3	25	75
Item7	Com1	5	30	150
Item8	Com1	3	10	30
Item9	Com2	2	25	50
Item10	Com3	4	30	120

Example: COUNT()

```
SELECT COUNT(*)
FROM PRODUCT_MAST;
```

Output:

10

Example: COUNT with WHERE

```
SELECT COUNT(*)
FROM PRODUCT_MAST;
WHERE RATE>=20;
```

Output:

7

Example: COUNT() with DISTINCT

```
SELECT COUNT(DISTINCT COMPANY)
FROM PRODUCT_MAST;
```

Output:

3

Example: COUNT() with GROUP BY

```
SELECT COMPANY, COUNT(*)  
FROM PRODUCT_MAST  
GROUP BY COMPANY;
```

Output:

Com1 5

Com2 3

Com3 2

Example: COUNT() with HAVING

```
SELECT COMPANY, COUNT(*)  
FROM PRODUCT_MAST  
GROUP BY COMPANY  
HAVING COUNT(*)>2;
```

Output:

Com1 5

Com2 3

2. SUM Function

Sum function is used to calculate the sum of all selected columns. It works on numeric fields only.

Syntax

```
SUM()  
  
or  
  
SUM([ALL|DISTINCT] expression)
```

Example: SUM()

```
SELECT SUM(COST)  
  
FROM PRODUCT_MAST;
```

Output:

```
670
```

Example: SUM() with WHERE

```
SELECT SUM(COST)  
  
FROM PRODUCT_MAST  
  
WHERE QTY>3;
```

Output:

```
320
```

Example: SUM() with GROUP BY

```
SELECT SUM(COST)  
  
FROM PRODUCT_MAST
```

```
WHERE QTY>3
```

```
GROUP BY COMPANY;
```

Output:

```
Com1    150
```

```
Com2    170
```

Example: SUM() with HAVING

```
SELECT COMPANY, SUM(COST)
```

```
FROM PRODUCT_MAST
```

```
GROUP BY COMPANY
```

```
HAVING SUM(COST)>=170;
```

Output:

```
Com1    335
```

```
Com3    170
```

3. AVG function

The AVG function is used to calculate the average value of the numeric type. AVG function returns the average of all non-Null values.

Syntax

```
AVG()
```

```
or
```

```
AVG([ALL|DISTINCT] expression)
```

Example:

```
SELECT AVG(COST)
FROM PRODUCT_MAST;
```

Output:

```
67.00
```

4. MAX Function

MAX function is used to find the maximum value of a certain column. This function determines the largest value of all selected values of a column.

Syntax

```
MAX()
or
MAX([ALL|DISTINCT] expression)
```

Example:

```
SELECT MAX(RATE)
FROM PRODUCT_MAST;
```

```
30
```

5. MIN Function

MIN function is used to find the minimum value of a certain column. This function determines the smallest value of all selected values of a column.

Syntax

```
MIN()
```

or

```
MIN([ALL|DISTINCT] expression)
```

Example:

```
SELECT MIN(RATE)
```

```
FROM PRODUCT_MAST;
```

Output:

```
10
```

```
*****
```

NULL values in SQL

In SQL there may be some records in a table that do not have values or data for every field and those fields are termed as a NULL value.

NULL values could be possible because at the time of data entry information is not available. So SQL supports a special value known as NULL which is used to represent the values of attributes that may be unknown or not apply to a tuple. SQL places a NULL value in the field in the absence of a user-defined value.

So, **NULL** values are those values in which there is no data value in the particular field in the table.

Importance of NULL Value

- It is important to understand that a NULL value differs from a zero value.
- A NULL value is used to represent a missing value, but it usually has one of three different interpretations:
 - The value unknown (value exists but is not known)
 - Value not available (exists but is purposely withheld)

- Attribute not applicable (undefined for this tuple)
- It is often not possible to determine which of the meanings is intended. Hence, SQL does not distinguish between the different meanings of NULL.

Principles of NULL values

- Setting a NULL value is appropriate when the actual value is unknown, or when a value is not meaningful.
- A NULL value is not equivalent to a value of ZERO if the data type is a number and is not equivalent to spaces if the data type is a character.
- A NULL value can be inserted into columns of any data type.
- A NULL value will evaluate NULL in any expression.
- Suppose if any column has a NULL value, then UNIQUE, FOREIGN key, and CHECK constraints will be ignored by SQL.

Test for NULL Values:

SQL allows queries that check whether an attribute value is NULL. Rather than using = or to compare an attribute value to NULL, SQL uses **IS** and **IS NOT**. This is because SQL considers each NULL value as being distinct from every other NULL value, so equality comparison is not appropriate.

Now, consider the following Employee Table.

Query:

```
CREATE TABLE Employee (Fname VARCHAR(50),Lname VARCHAR(50),SSN
VARCHAR(11),Phoneno VARCHAR(15),Salary FLOAT);
```

```
INSERT INTO Employee (Fname, Lname, SSN, Phoneno, Salary)
```

```
VALUES
```

```
('Shubham', 'Thakur', '123-45-6789', '9876543210', 50000.00),
```


('Aman', 'Chopra', '234-56-7890', NULL, 45000.00),
('Aditya', 'Arpan', NULL, '8765432109', 55000.00),
('Naveen', 'Patnaik', '345-67-8901', NULL, NULL),
('Nishant', 'Jain', '456-78-9012', '7654321098', 60000.00);

Output:

Fname	Lname	SSN	Phoneno	Salary
Shubham	Thakur	123-45-6789	9876543210	50000
Aman	Chopra	234-56-7890		45000
Aditya	Arpan		8765432109	55000
Naveen	Patnaik	345-67-8901		
Nishant	Jain	456-78-9012	7654321098	60000

The IS NULL Operator

Suppose we find the Fname and Lname of the Employee having no Super_ssn then the query will be:

Query:

SELECT Fname, Lname FROM Employee WHERE SSN IS NULL;

Output:

Fname	Lname
Aditya	Arpan

The IS NOT NULL Operator

Now if we find the Count of the number of Employees having SSNs.

Query:

SELECT COUNT(*) AS Count FROM Employee WHERE SSN IS NOT NULL;

Output:

Count
4

Updating NULL Values in a Table

We can update the NULL values present in a table using the UPDATE statement in SQL. To do so, we can use the IS NULL operator in the WHERE clause to select the rows with NULL values and then we can set the new value using the SET keyword.

Let's suppose that we want to update SSN in the row where it is NULL.

Query:

UPDATE Employee

SET SSN = '789-01-2345'

WHERE Fname = 'Aditya' AND Lname = 'Arpan';

select* from Employee;

Output:

Fname	Lname	SSN	Phoneno	Salary
Shubham	Thakur	123-45-6789	9876543210	50000
Aman	Chopra	234-56-7890		45000
Aditya	Arpan	789-01-2345	8765432109	55000
Naveen	Patnaik	345-67-8901		
Nishant	Jain	456-78-9012	7654321098	60000

Complex Integrity Constraints in SQL

Integrity constraints in SQL are rules that help ensure the accuracy and reliability of data in the database. They ensure that certain conditions are met when data is inserted, updated, or deleted.

While primary key, unique, and foreign key constraints are commonly discussed and used, SQL allows for more complex constraints through the use of CHECK and custom triggers. Here are some examples of complex integrity constraints:

1. Using CHECK Constraints

- Ensuring a range: You might want a column to only have values within a certain range.

Example:

```
CREATE TABLE Employees (  
  
    ID INT PRIMARY KEY,  
  
    Age INT CHECK (Age >= 18 AND Age <= 30)  
  
);
```

Pattern matching: Ensure data in a column matches a particular format.

Example:

```
CREATE TABLE Students (  
  
    ID INT PRIMARY KEY,  
  
    Email VARCHAR(255) CHECK (Email LIKE '%@%.%')  
  
);
```

2. Composite Primary and Foreign Keys

These are cases where the uniqueness or referential integrity constraint is applied over more than one column.

Example:

```
CREATE TABLE OrderDetails (  
  
    OrderID INT,  
  
    ProductID INT,  
  
    Quantity INT,  
  
    PRIMARY KEY (OrderID, ProductID),  
  
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),  
  
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID));
```

3. Using Stored Procedures

Sometimes, instead of direct data manipulation on tables, using stored procedures can help maintain more complex integrity constraints by wrapping logic inside the procedure. For instance, you could have a procedure that checks several conditions before inserting a record.

4. Triggers:

- Triggers are blocks of code that are automatically executed in response to specific events, such as insertions, updates, or deletions in a table.
- They can be used to enforce complex business rules or constraints that cannot be easily defined using standard SQL constraints.
- For example, you might use a trigger to update the last_updated timestamp whenever a row in a table is modified.
- A trigger is a procedure which is automatically invoked by the DBMS in response to changes to the database, and is specified by the database administrator (DBA).
- A database with a set of associated triggers is generally called an active database.

Parts of trigger

A triggers description contains three parts, which are as follows –

Event – An event is a change to the database which activates the trigger.

Condition – A query that is run when the trigger is activated is called as a condition.

Action – A procedure which is executed when the trigger is activated and its condition is true.

Use of trigger

Triggers may be used for any of the following reasons –

To implement any complex business rule, that cannot be implemented using integrity constraints.

Triggers will be used to audit the process. For example, to keep track of changes made to a table.

Trigger is used to perform automatic action when another concerned action takes place.

Types of triggers

The different types of triggers are explained below –

Statement level trigger – It is fired only once for DML statements irrespective of the number of rows affected by the statement. Statement-level triggers are the default type of trigger.

Before-triggers – At the time of defining a trigger we can specify whether the trigger is to be fired before a command like INSERT, DELETE, or UPDATE is executed or after the command is executed. Before triggers are automatically used to check the validity of data before the action is performed. For instance, we can use before trigger to prevent deletion of rows if deletion should not be allowed in a given case.

After-triggers – It is used after the triggering action is completed. For example, if the trigger is associated with the INSERT command then it is fired after the row is inserted into the table.

Create database trigger

To create a database trigger, we use the CREATE TRIGGER command. The details to be given at the time of creating a trigger are as follows –

- Name of the trigger.
- Table to be associated with.
- When trigger is to be fired: before or after.
- Command that invokes the trigger- UPDATE, DELETE, or INSERT.
- Whether row-level triggers or not.
- Condition to filter rows.
- PL/SQL block is to be executed when trigger is fired.

The syntax to create database trigger is as follows –

```
CREATE TRIGGER trigger_name
[BEFORE | AFTER]
[INSERT | UPDATE | DELETE]
ON table_name
[FOR EACH ROW | FOR EACH COLUMN]
TRIGGER body;
```

Example:

Creating Before Insert Trigger

Creating the Orders Table:

```
CREATE TABLE Orders (OrderID INT PRIMARY KEY, OrderDate DATE, TotalAmount
DECIMAL(10, 2));
```

```
CREATE TRIGGER
bi_order
BEFORE INSERT
ON
Orders
FOR EACH ROW
SET NEW.TotalAmount = NEW.TotalAmount * 5;
Insert into Orders values (101,'2023-08-10',500);
Insert into Orders values (102,'2023-07-15',400);
```

Insert into Orders values (103,'2023-05-03',300);

select * from orders;

OrderID	OrderDate	TotalAmount
101	2023-08-10	2500.00
102	2023-07-15	2000.00
103	2023-05-03	1500.00

Creating After Insert Trigger

Suppose we have created a table named "student_info" as follows:

```
CREATE TABLE student_info(stud_id int PRIMARY KEY,  
stud_code varchar (15) DEFAULT NULL,  
stud_name varchar (35) DEFAULT NULL,  
subj varchar (25) DEFAULT NULL,  
marks int DEFAULT NULL,  
phone varchar (15) DEFAULT NULL);
```

insert some records into this table

```
INSERT INTO student_info values(1,101,'Sai','DBMS',68,9912345670);  
INSERT INTO student_info values(2,102,'Varun','OS',70,9945675671);  
INSERT INTO student_info values(3,103,'Rahul','FLAT',90,9891075672);  
INSERT INTO student_info values(4,104,'Bharath','FLAT',90,9768975673);  
INSERT INTO student_info values(5,105,'Srinivas','FLAT',85,9345675674);  
INSERT INTO student_info values(6,106,'Kumar','SE',90,9761234675);  
INSERT INTO student_info values(7,107,'Sushanth','SE',83,9768123476);  
INSERT INTO student_info values(8,108,'Jayanth','SE',85,9768923477);  
SELECT * FROM STUDENT_INFO;
```

stud_id	stud_code	stud_name	subj	marks	phone
1	101	Sai	DBMS	68	9912345670
2	102	Varun	OS	70	9945675671
3	103	Rahul	FLAT	90	9891075672
4	104	Bharath	FLAT	90	9768975673
5	105	Srinivas	FLAT	85	9345675674
6	106	Kumar	SE	90	9761234675
7	107	Sushanth	SE	83	9768123476
8	108	Jayanth	SE	85	9768923477

Again, we will create a new table named "student_detail" as follows:

```
CREATE TABLE student_detail (stud_id int PRIMARY KEY,
stud_code varchar (15) DEFAULT NULL,
stud_name varchar (35) DEFAULT NULL,
subj varchar (25) DEFAULT NULL,
marks int DEFAULT NULL,
phone varchar (15) DEFAULT NULL,
Lasinserted Time);
```

Next, we will use a CREATE TRIGGER statement to create an **after_insert_details** trigger on the **student_info** table. This trigger will be fired after an insert operation is performed on the table.

Create Trigger after_insert_details

AFTER INSERT

ON

student_info

FOR EACH ROW

```
INSERT INTO student_detail VALUES (new. Stud_id, new. Stud_code,
new. stud_name, new. subj, new. marks, new. phone, CURTIME ());
```


Call the AFTER INSERT trigger:

We can use the following statements to invoke the above-created trigger:

INSERT INTO student_info values(9,109,'Sruthi','Java',67,9764567478);

The table that has been modified after the update query executes is student_detail. We can verify it by using the SELECT statement as follows:

stud_id	stud_code	stud_name	subj	marks	phone	Lasinserted
9	109	Sruthi	Java	67	9764567478	11:51:29
NULL	NULL	NULL	NULL	NULL	NULL	NULL

AFTER DELETE TRIGGER

```
create table student (id int primary key, name varchar (20), age int, Phone bigint);
```

```
insert into student Values(1,'Ravi',16,9912199121);
```

```
insert into student Values(2,'Seetha',18,9912199456);
```

```
insert into student Values(3,'Geetha',19,9912199789);
```

```
Select * from student;
```

AFTER DELETE TRIGGER

```
create table student (id int primary key, name varchar (20),age int, Phone bigint);
```

```
insert into student Values(1,'Ravi',16,9912199121);
```

```
insert into student Values(2,'Seetha',18,9912199456);
```

```
insert into student Values(3,'Geetha',19,9912199789);
```

```
Select * from student;
```

id	name	age	phone
1	Ravi	16	9912199121
2	Seetha	18	9912199456
3	Geetha	19	9912199789

create table student_backup (id int primary key, name varchar (20), age int, Phone bigint);

create trigger after_delete_student

after delete

on

student

for each row

insert into student_backup values (old.id,old.name,old.age,old.Phone);

delete from student where id=1;

select *from student_backup;

Result Grid				
	id	name	age	Phone
▶	1	Ravi	16	9912199121
✱	NULL	NULL	NULL	NULL

select *from student;

	id	name	age	Phone
▶	2	Seetha	18	9912199456
	3	Geetha	19	9912199789
✱	NULL	NULL	NULL	NULL

AFTER UPDATE TRIGGER

update student set name='Rishi' where id=2;

Select * from student;

id	name	age	phone
2	Rishi	18	9912199456
3	Geetha	19	9912199789

create trigger after_update_student

after update

on

student

for each row

insert into student_backup values(old.id,old.name,old.age,old.phone);

update student set name='Seetha' where id=2;

select *from student_backup;

Result Grid				
Filter Rows:				
	id	name	age	Phone
▶	1	Ravi	16	9912199121
	2	Rishi	18	9912199456
✱	NULL	NULL	NULL	NULL

select *from student;

id	name	age	Phone
2	Seetha	18	9912199456
3	Geetha	19	9912199789
NULL	NULL	NULL	NULL

Active databases.

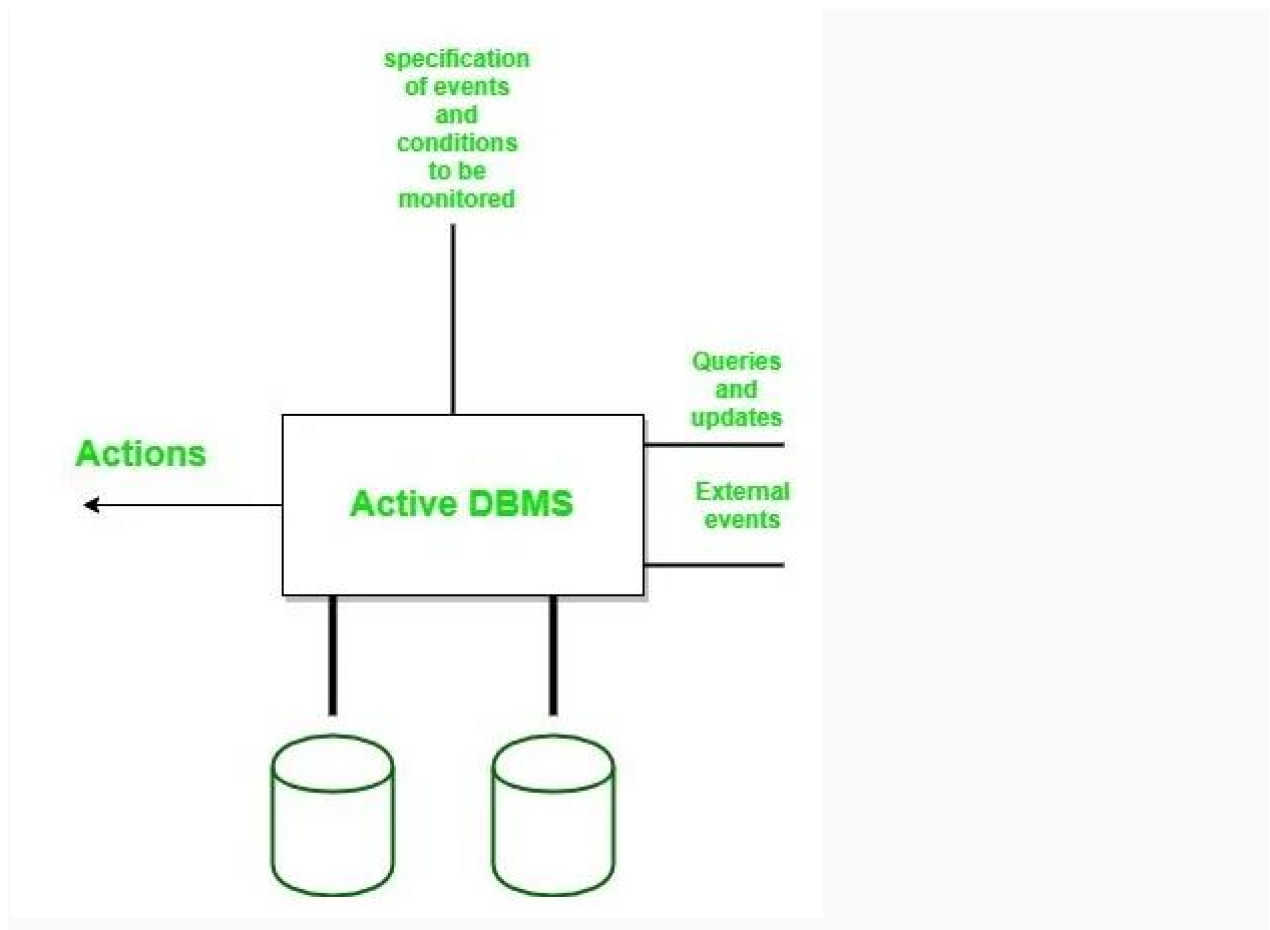
Active Database

The active database supports the preceding application by moving the reactive behavior from the application into the DBMS. Active databases are thus able to monitor and react to specific circumstances of relevance to an application. An active database system must provide a knowledge model i.e. description mechanism and an execution model for supporting this behavior. And for this we need **triggers** which have predefined action when situations occur.

An active Database is a database consisting of a set of triggers. These databases are very difficult to be maintained because of the complexity that arises in understanding the effect of these triggers. In such a database, DBMS initially verifies whether the particular trigger specified in the statement that modifies the database) is activated or not, prior to executing the statement.

If the trigger is active then DBMS executes the condition part and then executes the action part only if the specified condition is evaluated to true. It is possible to activate more than one trigger within a single statement.

In such a situation, DBMS processes each of the triggers randomly. The execution of an active part of a trigger may either activate other triggers or the same trigger that Initialized this action. Such types of triggers that activate themselves are called ‘recursive triggers’. The DBMS executes such chains of triggers in some predefined manner but it affects the concept of understanding.



Features of Active Database

- It possesses all the concepts of a conventional database i.e. data modeling facilities, query language, etc.
- It supports all the functions of a traditional database like data definition, data manipulation, storage management, etc.
- It supports the definition and management of ECA rules.
- It detects event occurrences.
- It must be able to evaluate conditions and execute actions.
- It means that it has to implement rule execution.

Advantages of Active Database

- It Enhances traditional database functionalities with powerful rule processing capabilities.
- Enable a uniform and centralized description of the business rules relevant to the information system.
- Avoids redundancy of checking and repair operations.
- A suitable platform for building a large and efficient knowledge base and expert systems.